CO5 – AT1

Course Title: Machine Learning Course Code: ITA0609

Coure faculty :DR. Shri Vindhya A

Student Name: K.Vishwanathan

Reg No:192312063

•

QUESTION 1 :

Two large Pandas DataFrames containing customer transaction and demographic data must be combined. However, constraints include: (i) no duplication of customer IDs, (ii) consistent indexing after merging, and (iii) preservation of all valid records. Describe how you will perform the combining operation, validate constraints using filtering and grouping, and ensure correctness through sorting and indexing techniques. Include steps to debug mismatched or missing entries.

1. **OBJECTIVE :**

The objective is to combine **customer transaction data** and **customer demographic data** using Pandas while:

- Avoiding **duplicate Customer IDs**
- Preserving **all valid records**
- Maintaining a **consistent index**
- Detecting **missing and mismatched entries**
- Validating the final merged DataFrame

```
import pandas as pd
transaction_data = {
   "Customer_ID": [101, 102, 103, 104, 105],
   "Amount": [5000, 3000, 4500, 7000, 2500],
   "Transactions": [5, 3, 4, 7, 2]
}
transactions = pd.DataFrame(transaction_data)

demographic_data = {
   "Customer_ID": [101, 102, 103, 105, 106],
   "Age": [25, 32, 28, 40, 35],
   "City": ["Chennai", "Madurai", "Coimbatore", "Salem", "Trichy"]
}

demographics = pd.DataFrame(demographic_data)

print("CUSTOMER TRANSACTION DATA")
print(transactions)

print("\nCUSTOMER DEMOGRAPHIC DATA")
print(demographics)
```

```
merged = pd.merge(
    transactions,
    demographics,
    on="Customer_ID",
    how="outer",
    indicator=True
)

print("\nMERGED DATA")
print(merged)


duplicates = merged[
    merged["Customer_ID"].duplicated(keep=False)
]

print("\nDUPLICATE CUSTOMER IDs")
print(duplicates)

duplicate_count = merged["Customer_ID"].duplicated().sum()

print("\nTotal Duplicate Customer IDs:", duplicate_count)


customer_count = merged.groupby(
    "Customer_ID"
).size()

print("\nCUSTOMER ID COUNT")
print(customer_count)

print("\nCUSTOMER IDs OCCURRING MORE THAN ONCE")
print(customer_count[customer_count > 1])


high_value_customers = merged[
    merged["Amount"] > 4000
]

print("\nCUSTOMERS WITH TRANSACTION AMOUNT > 4000")
print(high_value_customers)
```

```
    •
#
city_summary = merged.groupby(
   "City",
   dropna=False
)["Amount"].sum()

print("\nTOTAL TRANSACTION AMOUNT BY CITY")
print(city_summary)

sorted_data = merged.sort_values(
   "Amount",
   ascending=False
)

print("\nDATA SORTED BY TRANSACTION AMOUNT")
print(sorted_data)

sorted_data = sorted_data.reset_index(
   drop=True
)

print("\nDATA AFTER RESETTING INDEX")
print(sorted_data)


missing_demographic = merged[
   merged["_merge"] == "left_only"
]

print("\nCUSTOMERS MISSING DEMOGRAPHIC DATA")
print(missing_demographic)


missing_transaction = merged[
   merged["_merge"] == "right_only"
]

print("\nCUSTOMERS MISSING TRANSACTION DATA")
print(missing_transaction)


print("\nMISSING VALUES IN EACH COLUMN")
print(merged.isnull().sum())
```

```
print("\nFINAL VALIDATION")

print("Total Records:", len(sorted_data))

print(
    "Duplicate Customer IDs:",
    sorted_data["Customer_ID"].duplicated().sum()
)

print(
    "Missing Customer IDs:",
    sorted_data["Customer_ID"].isnull().sum()
)

print(
    "Continuous Index:",
    list(sorted_data.index)
)

print("\nFINAL RESULT")

if (
    sorted_data["Customer_ID"].duplicated().sum() == 0
    and
    sorted_data["Customer_ID"].isnull().sum() == 0
):
    print("Data validation successful.")
    print("No duplicate or missing Customer IDs found.")
else:
    print("Data validation requires further checking.")
```

Validation

The final DataFrame is checked for:

- **Record count**
- **Duplicate Customer IDs**
- **Missing Customer IDs**
- **Correct indexing**
- **Mismatched records**

12. COMPLETE OPERATION FLOW :

Two DataFrames

•
↓
MERGE
↓
VALIDATE
↓
FILTER
↓
GROUP
↓
SORT
↓
RESET INDEX
↓
DEBUG MISMATCHED / MISSING DATA
↓
FINAL VALIDATION

13. CONCLUSION

- **Outer merge** preserves valid records from both customer datasets.
- **Filtering and grouping** validate customer IDs and transaction information.
- **Sorting and resetting the index** provide a clean and consistent final DataFrame.
- **duplicated(), isnull(), and _merge** help debug duplicate, missing, and mismatched records.
- Thus, the combined customer dataset can be **validated and maintained accurately without unnecessary loss of valid records**.

QUESTION 2

Design a complete data analysis pipeline that reads multiple data files, processes them using Pandas DataFrames, and generates visualizations. Constraints include: (i) efficient file I/O operations, (ii) correct grouping and sorting of data, (iii) filtering based on user-defined conditions, and (iv) meaningful plotting outputs. Explain how each stage (loading, processing, aggregation, and plotting) will be implemented and optimized, along with debugging steps for handling corrupted files or inconsistent data formats.

1. OBJECTIVE

The objective is to design a complete **Pandas data analysis pipeline** that reads multiple files, processes the data, performs filtering, grouping, sorting and aggregation, and produces meaningful visualizations.
The pipeline must also handle **corrupted files, missing values and inconsistent data formats**.

2. DATASET

Two sales files are considered:

-
- sales_january.csv
- sales_february.csv

The dataset contains:

- **Product**
- **Region**
- **Sales**
- **Units**
- **Month**

PROGRAM :

```
import pandas as pd
import matplotlib.pyplot as plt

# Create January sales data
january_data = {
   "Product": ["Laptop", "Mobile", "Tablet", "Laptop", "Mobile"],
   "Region": ["Chennai", "Madurai", "Chennai", "Coimbatore", "Madurai"],
   "Sales": [50000, 30000, 20000, 45000, 35000],
   "Units": [5, 10, 6, 4, 12],
   "Month": ["January", "January", "January", "January", "January"]
}

january_df = pd.DataFrame(january_data)

# Create February sales data
february_data = {
   "Product": ["Laptop", "Mobile", "Tablet", "Laptop", "Mobile"],
   "Region": ["Chennai", "Madurai", "Chennai", "Coimbatore", "Madurai"],
   "Sales": [55000, 32000, 24000, 48000, 38000],
   "Units": [6, 11, 7, 5, 13],
   "Month": ["February", "February", "February", "February", "February"]
}

february_df = pd.DataFrame(february_data)

# Save the data as CSV files
january_df.to_csv("sales_january.csv", index=False)
february_df.to_csv("sales_february.csv", index=False)

# Read multiple files efficiently
files = ["sales_january.csv", "sales_february.csv"]
```

```
•
dataframes = []

for file in files:
    try:
        temp_df = pd.read_csv(file)
        dataframes.append(temp_df)
        print(file, "loaded successfully")

    except FileNotFoundError:
        print("File not found:", file)

    except pd.errors.EmptyDataError:
        print("File is empty:", file)

    except pd.errors.ParserError:
        print("Corrupted file:", file)

    except Exception as e:
        print("Error:", e)

# Combine all files
df = pd.concat(dataframes, ignore_index=True)

print("\nCOMBINED DATA")
print(df)

# Check data types
print("\nDATA TYPES")
print(df.dtypes)

# Check missing values
print("\nMISSING VALUES")
print(df.isnull().sum())

# Check duplicate records
print("\nDUPLICATE RECORDS")
print(df.duplicated().sum())

# Convert numerical columns into proper numeric format
df["Sales"] = pd.to_numeric(df["Sales"], errors="coerce")
df["Units"] = pd.to_numeric(df["Units"], errors="coerce")

# Remove duplicate records
```

```
df = df.drop_duplicates()

# User-defined filtering
sales_limit = 40000

filtered_data = df[df["Sales"] > sales_limit]

print("\nFILTERED DATA - SALES > 40000")
print(filtered_data)

# Grouping by Product
product_summary = df.groupby("Product")["Sales"].sum()

print("\nTOTAL SALES BY PRODUCT")
print(product_summary)

# Grouping by Region
region_summary = df.groupby("Region")["Sales"].sum()

print("\nTOTAL SALES BY REGION")
print(region_summary)

# Sorting products by total sales
sorted_products = product_summary.sort_values(
    ascending=False
)

print("\nPRODUCTS SORTED BY TOTAL SALES")
print(sorted_products)

# Aggregation
aggregation = df.groupby("Product").agg(
    Total_Sales=("Sales", "sum"),
    Average_Sales=("Sales", "mean"),
    Total_Units=("Units", "sum"),
    Average_Units=("Units", "mean")
)

print("\nPRODUCT-WISE AGGREGATION")
print(aggregation)

# Monthly sales
monthly_sales = df.groupby("Month")["Sales"].sum()
```

•

```
monthly_sales = monthly_sales.reindex(
    ["January", "February"]
)

print("\nMONTHLY SALES")
print(monthly_sales)

# Line plot
plt.figure(figsize=(8, 5))
plt.plot(
    monthly_sales.index,
    monthly_sales.values,
    marker="o"
)
plt.title("Monthly Sales")
plt.xlabel("Month")
plt.ylabel("Total Sales")
plt.grid(True)
plt.show()

# Bar chart
plt.figure(figsize=(8, 5))
plt.bar(
    sorted_products.index,
    sorted_products.values
)
plt.title("Total Sales by Product")
plt.xlabel("Product")
plt.ylabel("Total Sales")
plt.show()

# Histogram
plt.figure(figsize=(8, 5))
plt.hist(
    df["Sales"].dropna(),
    bins=5
)
plt.title("Sales Distribution")
plt.xlabel("Sales")
plt.ylabel("Frequency")
plt.show()
```

```
# Scatter plot
plt.figure(figsize=(8, 5))
plt.scatter(
    df["Units"],
    df["Sales"]
)
plt.title("Units Sold vs Sales")
plt.xlabel("Units Sold")
plt.ylabel("Sales")
plt.show()

# Final validation
print("\nFINAL VALIDATION")

print("Total Records:", len(df))

print(
    "Total Missing Values:",
    df.isnull().sum().sum()
)

print(
    "Total Duplicate Records:",
    df.duplicated().sum()
)

print("\nFINAL DATA")
print(df)
```

4. STAGE-WISE EXPLANATION

Loading

Multiple CSV files are read using **pd.read_csv()**.

try-except is used to detect **missing, empty or corrupted files**.

Processing

The files are combined using **pd.concat()**.

Missing values, duplicate records and incorrect numerical formats are checked before analysis.

Filtering

A user-defined condition is applied:

Sales > 40000

This extracts only high-value sales records.

Grouping

groupby() is used to calculate:

- **Total sales by product**

-
- **Total sales by region**

Sorting

sort_values() arranges products from **highest to lowest sales**, making comparison easier.

Aggregation

agg() calculates:

- **Total Sales**
- **Average Sales**
- **Total Units**
- **Average Units**

This gives a deeper numerical analysis of the dataset.

Visualization

The pipeline generates:

- **Line plot** → monthly sales trend
- **Bar chart** → product-wise sales comparison
- **Histogram** → sales distribution
- **Scatter plot** → relationship between units and sales

Debugging

The program handles:

- **FileNotFoundError** → file is unavailable
- **EmptyDataError** → file contains no data
- **ParserError** → corrupted/inconsistent CSV structure
- **Missing values** → detected using isnull()
- **Duplicate records** → detected using duplicated()
- **Incorrect numeric data** → handled using pd.to_numeric(..., errors="coerce")

16. COMPLETE PIPELINE

Multiple Files
↓
File Loading
↓
DataFrame Creation
↓
Data Processing
↓
Missing/Duplicate Checking
↓
User-Defined Filtering
↓
Grouping
↓
Sorting
↓

•

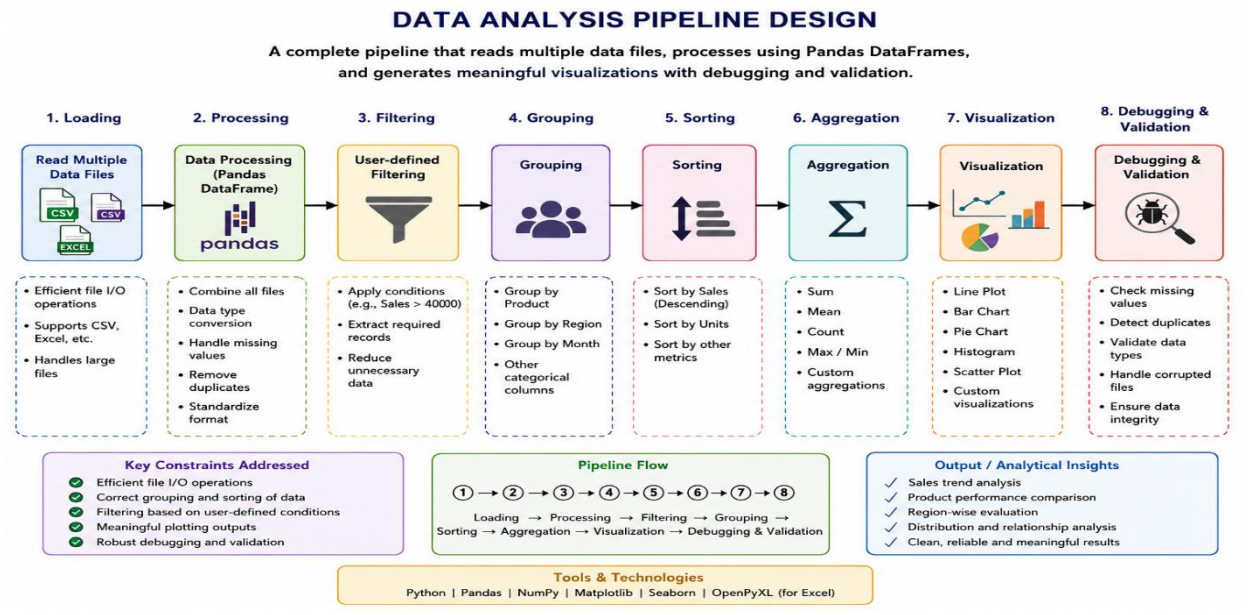
Aggregation
↓
Visualization
↓
Debugging
↓
Final Validation

6. ANALYTICAL RESULT

- The pipeline successfully combines **multiple sales files** into one DataFrame.
- **Filtering** identifies records above the selected sales limit.
- **Grouping and aggregation** provide product-wise and region-wise sales information.
- **Sorting** identifies the highest-performing products.
- **Visualizations** make trends, comparisons and relationships easier to understand.
- **Error handling and validation** improve the reliability of the analysis.

7. CONCLUSION

A complete **Pandas-based data analysis pipeline** is designed from file loading to final visualization.
The pipeline efficiently performs **processing, filtering, grouping, sorting and aggregation** while handling corrupted files and inconsistent data.



.